Image Processing - Exercise 1

Shay Morad

Introduction

The objective of this assignment was to design an algorithm that identifies scene transitions within provided video files. The method relies on analyzing cumulative histograms and detecting sharp, sudden changes between the histograms of consecutive frames. These abrupt shifts serve as indicators of a scene cut.

The two categories of videos exhibit notable differences. Videos in Category 1 consist of two segments separated by a single scene cut. In this case, the grayscale or color levels remain consistent throughout the frames, making the algorithm's task more straightforward. On the other hand, videos in Category 2 introduce an additional layer of complexity. While they also contain one scene cut, changes in grayscale or color levels occur, creating fluctuations in the histograms. Although these fluctuations are noticeable, they are not as prominent as those caused by an actual scene cut.

Algorithm

1. Load Video and Convert to Grayscale: The video is first loaded, and each frame is converted to grayscale to simplify the analysis by focusing on intensity variations rather than color differences.

2. Compute Cumulative Histogram: For each grayscale frame, a cumulative histogram is computed, which captures the distribution of pixel intensities across all pixels in the frame. This histogram is stored for later comparisons.

3. Calculate Differences Between Consecutive Frames: The algorithm calculates the difference between the cumulative histograms of consecutive frames using the L1 norm, which measures the total absolute difference between the two histograms. This difference quantifies how much the visual content has changed from one frame to the next.

4. Identify Scene Cuts: The frame pair with the largest difference (delta) between their cumulative histograms is identified as the most likely location of a scene cut. This is based on the assumption that scene transitions typically cause a sharp contrast in pixel intensity distributions, resulting in a significant jump in the delta.

5. Return Scene Cut Frames: The algorithm returns the frame with the largest delta and the next frame, indicating the point of scene transition.

In Category 2 videos, there are additional complexities introduced, such as minor resolution changes or brief video blurs within the same scene. While the basic algorithm remains the same, the method must differentiate between minor fluctuations in grayscale or color levels and actual scene transitions. The algorithm effectively detects these fluctuations but focuses on identifying the more significant changes, which correspond to scene cuts. Despite the fluctuations, the algorithm remains robust by identifying the largest delta between consecutive frames, ensuring it consistently detects true scene cuts even in the presence of these minor changes.

Implementation Details

First, the video was loaded using the read_video function from mediapy, which returned the video as a NumPy array. During this step, the output_format option ensured the video was automatically converted to grayscale. I then initialized an empty list to store cumulative histograms and processed each grayscale frame individually. For each frame, the histogram was calculated using numpy.histogram with 256 bins to represent the full range of grayscale values. This histogram was transformed into a cumulative histogram using numpy.cumsum and added to the list.

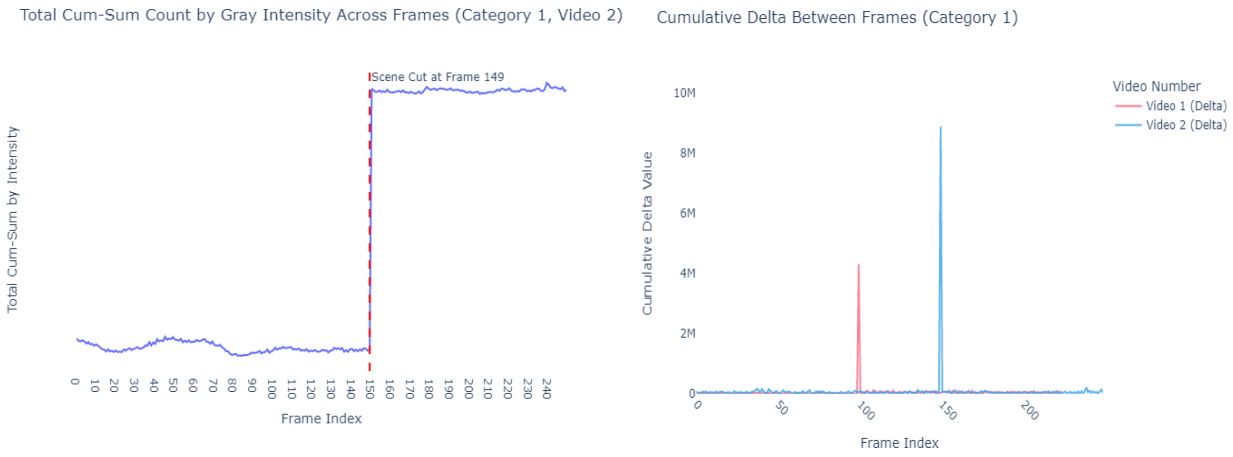
Next, I focused on calculating the differences between consecutive frames. Another empty list was prepared to hold these values. For every pair of cumulative histograms, I calculated the norm by subtracting the two histograms, applying numpy.abs to obtain the absolute differences, and then summing the values using numpy.sum. Each computed delta represented the magnitude of change between two consecutive frames and was stored in the list.

To pinpoint the frame with the most substantial change, I used numpy.argmax to locate the index of the largest delta. This index corresponded to the frame with the sharpest transition, signifying a scene cut. Finally, the algorithm returned the identified frame and the next frame as required by the assignment.

Most of the implementation is by using functionality from existing libraries, specifically mediapy and numpy as specified before. The only thing I wrote from "scratch" is the logic of it all while using external sources for help, such as reading the mediapy documentation, stackoverflow, LLM, google and more.
The only necessary "hyper-parameters" in the project I've set are the total amount of grayscale levels and the minimum and maximum levels, for the sake of pulling the wanted levels and calculating the norms we wanted as learnt in class.

Category 1 Results

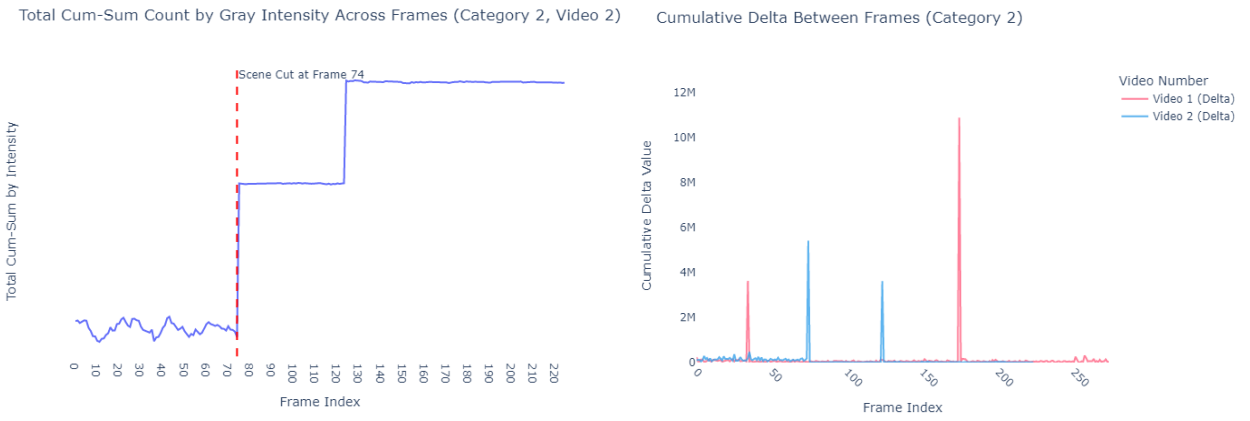
The following results are showcased on the files provided for category 1.



The left graph displays the cumulative sum of grayscale intensity across all pixels for video 2 in category 1. The graph shows a stable trend with minimal fluctuations until frames 149-150, where a significant spike occurs, indicating a scene change due to a noticeable shift in the distribution of grayscale levels. This observation is corroborated by the right graph, which illustrates the differences between consecutive frames. As seen in the blue graph, the differences remain consistently low throughout the video, with a sharp increase only at frames 149-150. This pattern aligns with the expectation that frames within the same scene exhibit minimal differences, whereas a scene cut results in a substantial change. Therefore, the scene cuts for the videos are identified as follows:
Video 1: between frames 99 and 100.
Video 2: between frames 149 and 150.

Category 2 Results

Category 2 differs from category 1 by not only featuring a scene cut but also a minor resolution change (or brief video blur) within the same scene. This results in a significant spike in the cumulative sum, as seen in category 1, occurring not only at the scene cut but also during the resolution change. The following graphs depict the same analysis as in category 1, but with the input videos for category 2.



The left graph displays video 2 from category 2, revealing two distinct spikes in the cumulative sum of gray intensity levels. One spike is significantly larger than the other, suggesting that the larger spike between frames 74-75 corresponds to a scene cut, while the smaller spike that follows indicates a resolution change. This interpretation is corroborated by the right graph, which shows the actual differences between consecutive frames. Here, the largest difference occurs between frames 74-75, aligning with the major spike observed on the left. Similarly, the red graph on the right confirms that the larger spike between frames 174-175 represents the scene cut.

Conclusion

In conclusion, the algorithm successfully detects scene transitions by analyzing cumulative histograms of grayscale intensity levels between consecutive frames and calculating the L1 norm of their differences. It accurately identifies scene cuts in Category 1 videos with consistent grayscale levels, and, despite the additional complexity of minor resolution changes in Category 2 videos, it still distinguishes between subtle fluctuations and actual scene cuts. This robustness ensures reliable scene detection across videos with varying visual characteristics, confirming the method's effectiveness in diverse scenarios. Overall, the approach provides a reliable and efficient solution for scene cut detection in video processing tasks.